Chapter 5. Tackling more complex problems with actor-critic methods

This chapter covers

- The limitations of the REINFORCE algorithm
- Introducing a *critic* to improve sample efficiency and decrease variance
- Using the advantage function to speed up convergence
- Speeding up the model by parallelizing training

In the previous chapter we introduced a vanilla version of a policy gradient method called REINFORCE. This algorithm worked fine for the simple CartPole example, but we want to be able to apply reinforcement learning to more complex environments. You already saw that deep Q-networks can be quite effective when the action space is discrete, but it has the drawback of needing a separate policy function such as epsilon-greedy. In this chapter you'll learn how to combine the advantages of REINFORCE and those of DQN to create a class of algorithms called actor-critic models. These have proven to yield state-of-the-art results in many domains.

The REINFORCE algorithm is generally implemented as an *episodic algorithm*, meaning that we only apply it to update our model parameters after the agent has completed an entire episode (and collected rewards along the way). Recall that the policy is a function, $\pi S \rightarrow P(a)$. That is, it's a function that takes a state and returns a probability distribution over actions (figure 5.1).

Figure 5.1. A policy function takes a state and returns a probability distribution over actions, where a higher probability indicates an action that is more likely to result in the highest reward.